FORMATION PLAYWRIGHT

Page Object Model

Structurer ses tests Playwright avec le pattern POM — Démonstration sur SauceDemo

Parcours	Framework	Pattern
Connexion → Produit → Détail	Playwright + TypeScript	Page Object Model

01 — CONTEXTE

Le problème que POM résout

Sans organisation, les tests automatisés deviennent rapidement **fragiles et impossibles à maintenir**. Imaginons un simple test de connexion :

```
// ❌ SANS POM — tout dans le test
test("connexion", async ({ page }) => {
  await page.goto("https://saucedemo.com");
  await page.fill("[data-test='username']", "standard_user");  // sélecteur en dur
  await page.fill("[data-test='password']", "secret_sauce");    // credential en dur
  await page.click("[data-test='login-button']");             // sélecteur en dur
  await expect(page).toHaveURL(/inventory/);
});
```

Le piège : si l'attribut `data-test='username'` change, il faut mettre à jour *chaque test* qui utilise ce sélecteur. Avec 50 tests, c'est un cauchemar.

Les symptômes d'un projet sans POM

Fragilité

- Un sélecteur change → 10 tests cassés
- Duplication massive du même code
- Difficile de savoir *où* corriger

Illisibilité

- Tests mélangeant *quoi tester* et *comment cliquer*
- Impossible de comprendre l'intention
- Onboarding d'un nouveau coéquipier très long

La solution : encapsuler chaque écran dans une classe

Le pattern **Page Object Model** consiste à créer **une classe TypeScript par page de l'application**. Cette classe expose des méthodes lisibles (`login()`, `clickFirstItem()`) et cache les sélecteurs. Les tests ne voient jamais les sélecteurs bruts.

02 — LE PATTERN

Les 4 piliers du POM

Encapsulation

Les sélecteurs sont **privés**. Seule la classe les connaît. Les tests ne voient que des méthodes.

Réutilisabilité

La méthode `login()` est écrite **une seule fois** et utilisée dans tous les tests qui en ont besoin.

Lisibilité

Un test se lit comme un scénario humain : *"je me connecte, je clique sur un produit, je vois le détail"*.

Maintenabilité

Si un sélecteur change, on ne modifie que **la classe**, jamais les tests. Un seul point de vérité.

Principe fondamental

Règle d'or : Un test ne doit *jamais* contenir de sélecteur CSS ou XPath. Il appelle uniquement les méthodes du Page Object. La logique de *comment interagir avec la page* appartient au POM, pas au test.

Structure d'un Page Object

```
export class LoginPage {
  // ① SÉLECTEURS — privés, jamais exposés aux tests
  private readonly usernameInput = this.page.getByPlaceholder("Username");
  private readonly passwordInput = this.page.getByPlaceholder("Password");
  private readonly loginButton   = this.page.getByRole("button", { name: "Login" });

  constructor(private readonly page: Page) {}

  // ② ACTIONS — méthodes publiques, verbes à l'infinitif
  async goto() {
    await this.page.goto("/");
  }

  async login(username: string, password: string) {
    await this.usernameInput.fill(username);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }

  // ③ ASSERTIONS — vérifications d'état
  async assertOnLoginPage() {
    await expect(this.page).toHaveURL("/");
```

```
    }
  }
```

Chaque Page Object contient trois zones : les **sélecteurs** (privés), les **actions** (ce qu'on peut faire sur la page), et les **assertions** (ce qu'on peut vérifier).

03 — PARCOURS DÉMONTRÉ

Le scénario couvert

Ce projet illustre un parcours minimaliste mais complet : **trois écrans, deux Page Objects, un test de bout en bout**.

Le test en clair

```
// tests/inventory.spec.ts
test.describe("Feature: Détail d'un produit", () => {

  test.beforeEach(async ({ loginPage }) => {
    await loginPage.goto();
    await loginPage.login(USERS.standard.username, USERS.standard.password);
  });

  test("Cliquer sur le premier produit affiche sa page de détail", async ({
    inventoryPage, inventoryItemPage
  }) => {
    // Given : connecté sur la liste des produits (beforeEach)
    await inventoryPage.assertOnInventoryPage();

    // When : je clique sur le premier produit
    await inventoryPage.clickFirstItem();

    // Then : je vois la page de détail
    await inventoryItemPage.assertOnItemDetailPage();
  });

});
```

À noter : Le test ne connaît aucun sélecteur. Il exprime uniquement l'*intention* : connecter, cliquer, vérifier. Le *comment* est dans les Page Objects.

Les deux tests du projet

Fichier	Scénario	Page Objects utilisés
`login.spec.ts`	Connexion réussie → redirection /inventory	`LoginPage`, `InventoryPage`
`inventory.spec.ts`	Clic premier produit → page de détail	`InventoryPage`, `InventoryItemPage`

04 — ARCHITECTURE

Structure du projet

```
Projet-POM-Saucedemo/ ├── src/ │ ├── pages/ # ← Page Objects (un fichier par écran)
│ │ ├──        LoginPage.ts        # Page de connexion │ │ ├──
       InventoryPage.ts        # Liste des produits │ │ └──
       InventoryItemPage.ts        # Détail d'un produit │ ├── fixtures/ │ │ └──
index.ts # Injection automatique des POMs dans les tests │ └── data/ │ └── users.ts
# Données de test centralisées ├── tests/ # ← Fichiers de test │ ├── login.spec.ts
# Test : authentification │ └── inventory.spec.ts # Test : navigation vers le
détail └── playwright.config.ts # Configuration (baseURL, slowMo, reporter…)
```

Principe de séparation

src/pages/ — Le COMMENT

- Contient les sélecteurs CSS
- Contient les actions bas niveau
- Peut changer si l'UI change
- Ne contient aucune logique de test

tests/ — Le QUOI

- Exprime l'intention métier
- Ne contient aucun sélecteur
- Lisible par un non-développeur
- Stable même si l'UI change

Les fixtures : injection automatique

Plutôt que d'instancier chaque POM dans chaque test (`new LoginPage(page)`), on utilise un système de **fixtures personnalisées**. Playwright injecte automatiquement les objets dans les tests via leur nom.

```
// src/fixtures/index.ts
export const test = base.extend<Pages>({
  loginPage:         async ({ page }, use) => use(new LoginPage(page)),
  inventoryPage:     async ({ page }, use) => use(new InventoryPage(page)),
  inventoryItemPage: async ({ page }, use) => use(new InventoryItemPage(page)),
});

// Dans un test, on déstructure simplement :
test("mon test", async ({ loginPage, inventoryPage }) => { ... });
```

Détail des 3 Page Objects

1 — LoginPage `src/pages/LoginPage.ts`

Responsabilité : authentification sur /index.html

- **Sélecteurs :** champ Username, champ Password, bouton Login
- `goto()` → navigue vers la racine du site
- `login(username, password)` → remplit le formulaire et soumet
- `assertOnLoginPage()` → vérifie que l'URL est `/`

2 — InventoryPage `src/pages/InventoryPage.ts`

Responsabilité : liste produits sur /inventory.html

- **Sélecteurs :** titre de page, noms des produits
- `clickFirstItem()` → clique sur le nom du premier produit
- `assertOnInventoryPage()` → vérifie URL (`/inventory.html`) et titre "Products"

3 — InventoryItemPage `src/pages/InventoryItemPage.ts`

Responsabilité : détail produit sur /inventory-item.html

- **Sélecteurs :** nom de l'article affiché
- `assertOnItemDetailPage()` → vérifie URL (`/inventory-item.html`) et nom visible

Comparaison avec et sans POM

Critère	Sans POM	Avec POM
Sélecteur change	✗ Modifier N tests	✓ Modifier 1 classe
Lisibilité du test	✗ Sélecteurs cryptiques	✓ Méthodes expressives
Onboarding	✗ Comprendre chaque sélecteur	✓ Lire les noms de méthodes
Duplication	✗ Copier-coller partout	✓ Appel unique à la méthode
Ajout d'un test	✗ Réécrire les interactions	✓ Réutiliser les méthodes

Lancer le projet

1 Installer les dépendances

Installe Playwright et les navigateurs nécessaires.

```
npm install
npx playwright install chromium
```

2 Lancer les tests (mode visible)

Ouvre le navigateur avec un ralentissement de 1 seconde pour observer chaque action.

```
npm run test:headed
```

3 Consulter le rapport

Ouvre le rapport HTML avec captures d'écran en cas d'échec.

```
npm run report
```

Ce que vous observerez

Test 1 — login.spec.ts

- Naviguer vers `saucedemo.com`
- Saisir `standard_user` / `secret_sauce`
- Cliquer sur "Login"
- Vérifier l'URL `/inventory.html`

Test 2 — inventory.spec.ts

- Connexion automatique (beforeEach)
- Vérifier la page produits
- Cliquer sur le 1er produit
- Vérifier `/inventory-item.html`

L'essentiel : le POM ne change pas ce que les tests font — il change *comment le code est organisé*. Les tests restent clairs, les Page Objects absorbent la complexité technique.